Bevy Engine

Augsburg Rust Meetup · Road to Game Jam 2026

Über diesen Vortrag

Einführung in Bevy für unseren Workshop bei TUXEDO Computers (24.–26. April) im Rahmen der **Road to Game Jam 2026**.

- Was Bevy ist, warum es spannend ist, was ECS bedeutet
- Wir schauen uns **destroy.rs** an, einen Arkanoid-Klon
- Fragen jederzeit willkommen

Was ist Bevy?

- **Datengetriebene Game Engine** in Rust
- Frei und quelloffen · MIT/Apache 2.0
- Version **0.18** (Januar 2026) · 2D und 3D
- Rein code-basiert, schnelle Hot Reloads (0,8–3s)
- Wachsendes Ökosystem an Plugins und Crates

Warum Bevy?

Für Rustaceans

Spiellogik wie Systemcode.
Keine Skriptsprache, nur Rust.

Für Spieleentwickler

ECS-Architektur, die skaliert.
Standardmäßig parallel.

Für Tüftler

Modularer Aufbau. Alles ist ein Plugin, austauschbar.

Für Open Source

Bevy Foundation. 174 Contributors allein für 0.18.

Entity Component System

Das Problem mit OOP

Tiefe Vererbungsbäume:

```
GameObject → Character → Enemy → FlyingEnemy → FlyingBoss
GameObject → Character → Player → PlayerWithShield
```

Was, wenn ein FlyingBoss einen Schild braucht?
Vererbung lässt sich schlecht kombinieren.

ECS — Die große Idee

Trenne **Identität**, **Daten** und **Verhalten**:

Entity

Nur eine ID (wie eine Datenbankzeile)

Component

Daten an Entities (Rust-Structs)

System

Logik auf Components (Rust-Funktionen)

ECS ist wie eine Tabelle!

Components

Entities

A	B				E
		Position	Velocity	Health	Sprite
1		10	2	100	player
2		5	3	85	
3		8	7	65	
4			4	65	enemy1
5		7	5	15	
6		3		30	enemy3
7		3		90	

Systems

=AVG(B2:B7)

=SUM(C2:C7)

Zeilen = Entities · Spalten = Components · Systems = Formeln neben der Tabelle

Entities & Components

```
#[derive(Component)]
struct Health(u32);

#[derive(Component)]
struct Player;  // "Marker" — keine Daten

// Entity spawnen (= Zeile zur Tabelle)
commands.spawn((
    Player, Health(100),
    Position { x: 0.0, y: 0.0 },
    Sprite::default(),
));
```

Systems

Systems sind Rust-Funktionen. Bevy ruft sie auf.

```
fn movement(mut q: Query<(&Velocity, &mut Position)>) {
    for (vel, mut pos) in &mut q {
        pos.x += vel.x;
        pos.y += vel.y;
    }
}
```

Bevy parallelisiert Systems automatisch, wenn es sicher ist.

Komposition statt Vererbung

```
// Fliegender Boss mit Schild – einfach kombinieren!
commands.spawn((
    Enemy, Flying { altitude: 50.0 },
    Shield { hp: 30 }, Health(200), Boss,
));
```

Spieler soll fliegen? `Flying` hinzufügen.
Boss ohne Schild? `Shield` weglassen. **Frei kombinierbar.**

Queries — Entities auswählen

Queries sind wie SQL-`SELECT` für die Spielwelt. Ein System liest/schreibt nur über die Zeilen, die passen.

```
// Alle Entities mit Health und Position
fn show_health(q: Query<(&Health, &Position)>) { ... }

// Nur Entities mit Enemy UND Flying
fn fly_enemies(
    q: Query<&mut Position, (With<Enemy>, With<Flying>)>,
) { ... }
```

ECS hat Bevy nicht erfunden

- Das Konzept stammt aus den frühen 2000ern (Scott Bilas, GDC 2002)
- Populär durch den Overwatch-Vortrag (GDC 2017) und Unity DOTS
- Heute in vielen Engines und Frameworks verbreitet

Bevy baut auf einem bewährten Konzept auf, das es nicht erfunden hat, aber besonders elegant in Rust umsetzt.

Zwei Arten von ECS

Archetype-basiert

Entities mit gleichen Components werden in Tabellen (Archetypes) gruppiert. Sehr schnelle Iteration, aber Umzug beim Hinzufügen/Entfernen von Components.

Bevy, Unity DOTS, Flecs

Sparse Set

Jeder Component-Typ hat seinen eigenen Speicher. Schnelles Hinzufügen/Entfernen, aber Iteration über mehrere Components ist aufwendiger.

EnTT (C++, Minecraft Bedrock), Shipyard

Bevy nutzt Archetypes. In der Praxis ist der Unterschied für die meisten Spiele kaum spürbar.

Bevy-Architektur

Die App & Plugins

```rust
use bevy::prelude::*;

fn main() {
    App::new()
        .add_plugins(DefaultPlugins)
        .add_systems(Startup, setup)
        .add_systems(Update, (movement, collision))
        .run();
}
```

- `DefaultPlugins` = Fenster, Rendering, Input, Audio, …
- Alles ist ein Plugin, austauschbar

Resources & Events

Resources

Globale Daten · Punktestand, Timer.

```
#[derive(Resource)]
struct Score(u32);
```

Events

Entkoppeln Systems. Senden & Empfangen.

```
#[derive(Event)]
struct HitEvent;
```

States & Schedules

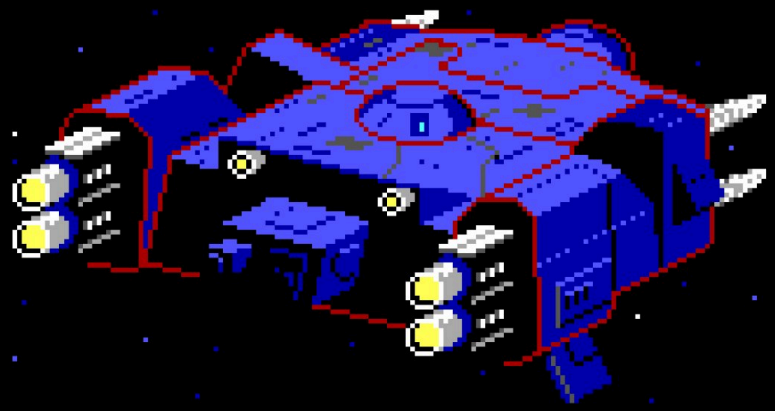
```rust
#[derive(States, Default, Clone, PartialEq, Eq, Hash)]
enum GameState {
    #[default] Menu,
    Playing,
    Paused,
}
```

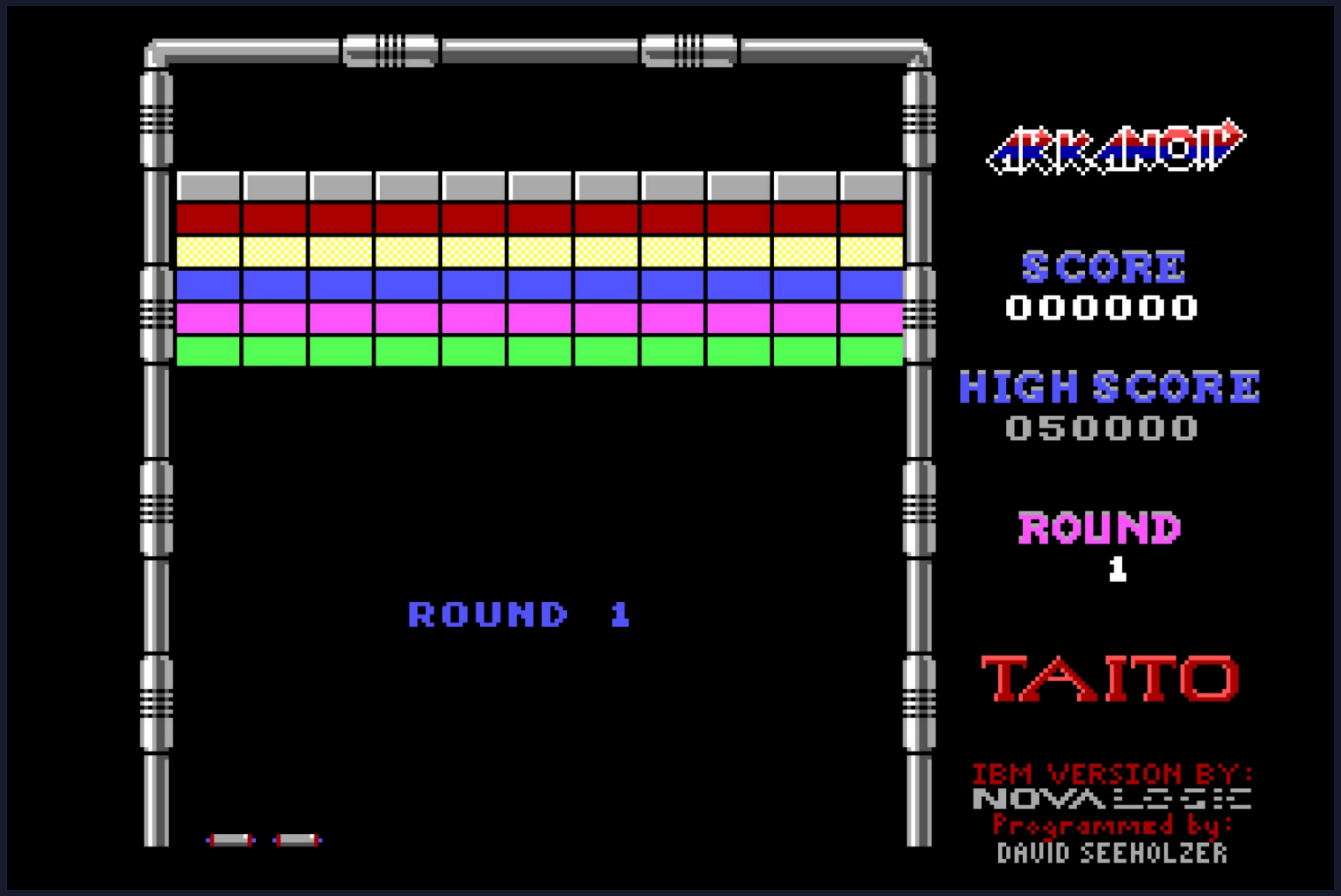
- Systems an States gebunden, laufen nur im aktiven State
- Schedules: `Startup`, `Update`, `FixedUpdate`, `OnEnter`, `OnExit`

destroy.rs — Ein 2D-Spiel bauen

Arkanoid (Taito, 1986) · das Original, das wir nachbauen

BUT ONLY TO BE TRAPPED IN SPACE
BY SOMEONE ....




ARKANOID
SCORE
000000
HIGH SCORE
050000
ROUND
1
ROUND 1
TAITO
IBM VERSION BY:
NOVALOGIC
Programmed by:
DAVID SEEHOLZER

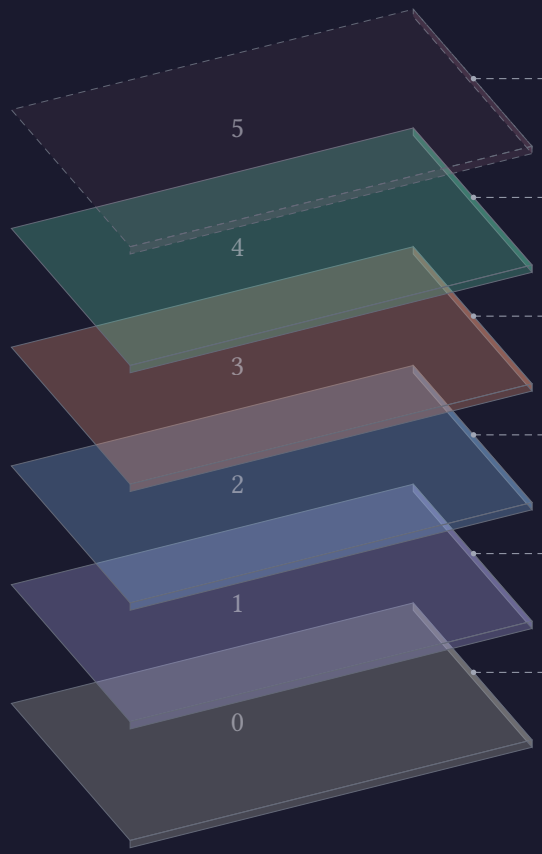
Warum dieses Spiel?

Warum 2D?

Kein Kampf mit 3D-Meshes. Fokus auf Bevy, nicht Modelling. Trotzdem „fast 3D“ dank Blender-Rendering.

Warum Breakout?

Klassiker wie Tetris sind markenrechtlich geschützt. Breakout/Arkanoid ist als Konzept frei nutzbar.



FX
Beauty Pass · kein echter Layer

UI
HUD, Punkte, Menüs

Particle
Trümmer, Funken, Effekte

Sprite
Alle Spielobjekte

Shadow
Vorgerenderte Objektschatten

BG
Gekachelte Textur oder Shader

Sprites in Bevy

```
// Sprite laden
commands.spawn((
    Sprite::from_image(asset_server.load("vaus.png")),
    Transform::from_xyz(0.0, -200.0, 0.0),
));

// Horizontal spiegeln — keine zweite Textur nötig
commands.spawn(Sprite {
    image: asset_server.load("enemy.png"),
    flip_x: true, ..default()
});
```

`flip_x`/`flip_y`, Einfärbung, Z-Sortierung, Spriteatlanten · alles eingebaut.

Normal Maps — Beleuchtete Sprites

Bevy unterstützt **2D-Beleuchtung mit Normal Maps**. Eine Normal Map neben dem Diffuse-Sprite, und Bevy beleuchtet ihn dynamisch. Sprites reagieren auf Licht, ohne 3D-Mesh.

Für destroy.rs gehen wir noch weiter: Rendern in **Blender 3D** mit orthografischer Kamera. Volle GI und Reflexionen direkt in den Sprite eingebacken. Ergebnis: „fast 3D“, aber technisch ein flaches Bild.

Input & Kollision

```
fn paddle_input(
    keyboard: Res<ButtonInput<KeyCode>>,
    mut q: Query<&mut Velocity, With<Paddle>>,
) {
    let mut vel = q.single_mut();
    vel.x = 0.0;
    if keyboard.pressed(KeyCode::ArrowLeft)  { vel.x =
-300.0; }
    if keyboard.pressed(KeyCode::ArrowRight) { vel.x =
300.0; }
}
```

Physik: Avian (Community-Standard) oder eigene Checks.

Sound — Was funktioniert

```
commands.spawn((
    AudioPlayer(asset_server.load("hit.ogg")),
    PlaybackSettings::DESPAWN,
));
```

- OGG, WAV, FLAC, MP3 · Looping, Lautstärke, Spatial Audio
- Sound als Entity, passt perfekt zu ECS

Sound — Die Lücke

- **Keine Mixer-Busse, kein EQ, kein Kompressor**
- Keine Filter, keine Effektkette, kein FMOD/Wwise
- Pitch = Abspielgeschwindigkeit (Chipmunk-Effekt)

Der alte Trick: C64 (SID, 3 Stimmen), Amiga (Paula, 4 Stimmen) — feste Stimmenzahl = kein Clipping. Bevy hat keinen Kompressor. Stimmenzahl selbst verwalten oder Verzerrung akzeptieren.

Bevys Schwächen

- **Audio** · `bevy_audio` ist sehr einfach. `bevy_kira_audio` hilft, reicht aber nicht ganz.
- **API-Brüche** ca. alle 3 Monate · Upgrades sind echte Arbeit
- **Kein Editor** · alles Code, kein visuelles Werkzeug
- **Doku ist dünn** · Quellcode + Discord für Fortgeschrittenes

Bevys Schwächen (Forts.)

- **Keine Konsolen** · kein PlayStation, Xbox, Switch
- **Web-Builds** · kein Multithreading, WebGL2, Audio-Glitches
- **Kalte Kompilierung** · erster Build dauert Minuten
- **Junges Ökosystem** · weniger Plugins als Godot/Unity

Ökosystem

avian2d/avian3d

Physik · Collider, Rigid Bodies, Joints

bevy_kira_audio

Erweiterte Audio-Wiedergabe & Mixing

bevy_egui

Immediate-Mode-UI für Debug-Tools

bevy_ecs_ldtk

LDtk-2D-Level-Editor-Maps laden

Lernressourcen

- **Bevy Book** · offizielle Einführung
- **Bevy Cheat Book** · Community-Referenz
- **Discord** · sehr aktiv, anfängerfreundlich
- **Bevy Jams** · regelmäßige Game Jams
- **This Week in Bevy** · wöchentlicher Newsletter

Der Workshop

Was wir weglassen

Kein echtes 3D

Blender-Pipeline liefert „fast 3D“.

Kein Level-Loading

Levels im Code definiert.

Keine Config

Hardcoded. Recompile ist okay.

Kein Speichern

Keine Serialisierung.

Keine Controller

Nur Tastatur.

Kein Netzwerk

Nur Einzelspieler.

Auch kein **Scrolling** (festes Spielfeld), keine Partikel, keine Animationen, kein Web-Export.

Der Workshop — 24.–26. April

Tag 1
Fr 16–19 Uhr

Setup & Einführung
ECS im Detail
„Hello Bevy“

Tag 2
Sa 9–17 Uhr

2D-Spiel bauen
Sprites, Input, Kollision
UI & Punktestand

Tag 3
So 9–15 Uhr

Fortgeschrittene Themen
Freie Projektzeit
Showcase & Fragen

Ort: TUXEDO Computers, Augsburg **Plätze:** max. 15 **Kosten:** Kostenlos

Was ihr braucht

- Grundlegende Rust-Kenntnisse
- Laptop mit **Rust-Toolchain**
- Bildbearbeitungsprogramm (GIMP, Krita, Aseprite)
- Neugier und Experimentierfreude

Ausblick

Echtes 3D

3D-Meshes, PBR-Materialien, Echtzeit-Beleuchtung.

Pixel-Art-Level

Aseprite/GIMP ⟶ Tilemaps in Bevy.

KI-generierte Level

Midjourney/Flux ⟶ komplettes Level-Thema.

Game Jam — 8.–10. Mai!

TUXEDOs erster Game Jam. Mitmachen!

Wir sehen uns beim Workshop!

24.–26. April 2026 · TUXEDO Computers, Augsburg

Organisiert vom **Augsburg Rust Meetup**
in Zusammenarbeit mit **TUXEDO Computers**

Michael Schury · **Tiago Manczak** · **Ludwig Weinzierl**

Danke an Michael und Tiago, die diesen Workshop aktiv vorangetrieben haben!